Research Analysis

Job Sequencing and Tool Switching Problem

Heuristic Bounds, Grouping Selection, and Collapse Variants

Three-Part Scientific Analysis

Part I: Worst-Case Gap Analysis
Part II: Grouping Selection — Mathematical and ML Directions
Part III: SSP Variants that Collapse to JGP

Scope. This document addresses three original research questions in the Uniform Scheduling and Sequencing Problem (SSP) for Flexible Manufacturing Systems (FMS). The analysis establishes tight worst-case bounds on a configuration-based TSP heuristic, proposes exact and heuristic methods (mathematical and ML-based) for grouping selection, and characterises SSP variants whose optimal solutions coincide with those of the Job Grouping Problem (JGP).

Contents

1 Global Notation and Problem Setup **2**

2 Part I: Worst-Case Gap Between the Heuristic and $\mathrm{OPT}_{\mathrm{SSP}}$ **2**
2.1 The Two Sources of the Gap . . . 2
2.2 Lower Bounds on $\mathrm{OPT}_{\mathrm{SSP}}$ . . . 2
2.3 Construction I: Disjoint Private Tool Sets . . . 3
2.4 Construction II: Overlapping Mandatory Tools — The Tight Construction . . . 4
2.4.1 Heuristic Cost (Naïve Padding) . . . 4
2.4.2 Optimal SSP Cost (Lookahead Configuration) . . . 4
2.4.3 Proof That (2) Is Indeed the Optimum . . . 5
2.5 The Tight Gap Bound . . . 6
2.6 Structural Insights and Summary of Part I . . . 6

3 Part II: Grouping Selection for Optimal and Heuristic SSP Solving **6**
3.1 Mathematical and Logical Directions . . . 6
3.1.1 Reduction to Generalized TSP (GTSP) — The Exact Formulation . . . 6
3.1.2 Grouping Selection via Tool-Overlap Graph . . . 7
3.1.3 Lagrangian Relaxation — Exploring the Pareto Frontier . . . 8
3.1.4 Column Generation — The Exact Master Problem . . . 8
3.1.5 Tool-Conflict Hypergraph and Chromatic Structure . . . 9
3.2 Machine Learning Directions . . . 9
3.2.1 Graph Neural Network (GNN) Grouping Scorer . . . 9
3.2.2 Reinforcement Learning for Joint Grouping and Sequencing . . . 10
3.2.3 Learning-Augmented Column Generation . . . 10
3.2.4 Contrastive Learning for Grouping Comparison . . . 11

4 Part III: SSP Variants That Collapse to JGP **11**
4.1 Formal Definition of Collapse . . . 11
4.2 The Baseline Collapse: Switching Instants (0/1 Cost) . . . 11
4.3 Threshold SSP . . . 12
4.4 Per-Tool Weight Variant . . . 12
4.5 General Collapse Characterisation . . . 13
4.6 A Novel Variant: Setup-Matrix SSP . . . 13
4.7 Machine Learning for Collapse Variants . . . 14
4.7.1 Learning the Collapse Threshold θ^* . . . 14
4.7.2 Meta-Learning a Collapse-Inducing Cost Function . . . 14
4.7.3 Instance-Specific Lagrangian Parameter Learning . . . 14

5 Summary of Key Results **15**

6 Open Questions **15**

Global Notation and Problem Setup

The following notation is fixed throughout. Let:

- $J = \{1, \ldots, n\}$: set of n jobs; $T = \{1, \ldots, m\}$: set of m tools.
- $C \in \mathbb{Z}_{>0}$: magazine capacity (number of tool slots).
- $T_j \subseteq T$, $|T_j| \leq C$: tools required by job j.
- A **configuration** $\mathcal{C}_k \subseteq T$ with $|\mathcal{C}_k| = C$ is a feasible magazine load such that $\bigcup_{j \in G_k} T_j \subseteq \mathcal{C}_k$ for some group G_k.
- $\mathfrak{C}(G_k) = \left\{\mathcal{C} \subseteq T : \bigcup_{j \in G_k} T_j \subseteq \mathcal{C},\ |\mathcal{C}| = C\right\}$: the set of all valid configurations for group G_k.
- A **grouping** $\mathcal{G} = \{G_1, \ldots, G_K\}$ is a partition of J into K groups, each coverable by some configuration.
- K^*: the JGP-optimal (minimum) number of groups.
- $w(\mathcal{C}, \mathcal{C}') = |\mathcal{C} \setminus \mathcal{C}'|$: switching cost (tools removed when transitioning from $\mathcal{C}$ to $\mathcal{C}'$); equals tools inserted by feasibility of the uniform model.
- $\mathrm{OPT}_{\mathrm{SSP}}$: optimal SSP solution cost (path, unless stated otherwise).
- H: cost of the heuristic (JGP-optimal grouping + TSP on configurations).
- $s_k = C - |T_{G_k}|$: **padding slack** of group G_k (free magazine slots that can be filled with any tools).

The heuristic. The heuristic under study proceeds as: (1) solve JGP to obtain K^* groups; (2) for each group G_k, select a configuration $\hat{\mathcal{C}}_k \in \mathfrak{C}(G_k)$ (the "canonical" or "naïve" configuration—padding with tools not from adjacent groups); (3) solve TSP on the K^* configurations with arc weight $w(\cdot, \cdot)$; (4) read off the SSP path from the TSP tour. We seek to bound $H - \mathrm{OPT}_{\mathrm{SSP}}$.

Part I: Worst-Case Gap Between the Heuristic and $\mathrm{OPT}_{\mathrm{SSP}}$

The Two Sources of the Gap

The gap $H - \mathrm{OPT}_{\mathrm{SSP}} \geq 0$ (since the heuristic is feasible) arises from two independent sources.

- **Source 1 — Configuration choice.** For a *fixed* grouping, different padding choices for each group yield different switching costs between adjacent groups. The heuristic uses a specific (possibly non-optimal) configuration per group. The SSP optimum may use a different configuration for the same group, strategically padding toward adjacent groups' mandatory tools. This source is eliminated by solving a Generalized TSP (GTSP) over all valid configurations per group (see section 3.1.1).
- **Source 2 — Grouping choice.** The JGP-optimal grouping with K^* groups need not minimize switching cost. The SSP optimum may use $K > K^*$ groups (paying one extra inter-group transition) while achieving lower per-transition cost through greater tool overlap. This is the dominant and theoretically more interesting source.

In the rest of Part I we concentrate on Source 1, since it is the source that the specific heuristic described (fixed canonical padding + TSP) introduces, and we construct instances where the gap is arbitrarily large.

Lower Bounds on $\mathrm{OPT}_{\mathrm{SSP}}$

Lemma 2.1 (Global Switching Lower Bound [6])**.** *For any instance of the SSP,*

$$\mathrm{OPT}_{SSP} \ \geq \ m - C,$$

where $m = |T|$ is the total number of distinct tools.

Proof. At the start of the job sequence the magazine holds C tools. Every tool $t \in T$ must be loaded at least once to be used. The C tools already present need not be loaded again, so at least $m - C$ tool insertions are required. Since the uniform model pairs every insertion with a removal (magazine stays full), the total number of switches is at least $m - C$. □ □

The global bound is not always tight enough for our construction; we need a per-transition bound.

Lemma 2.2 (Per-Transition Lower Bound)**.** *For any two consecutive groups G_k, G_{k+1} in any grouping, and for any valid configurations $\mathcal{C}_k \in \mathfrak{C}(G_k)$, $\mathcal{C}_{k+1} \in \mathfrak{C}(G_{k+1})$,*

$$w(\mathcal{C}_k, \mathcal{C}_{k+1}) \;\geq\; \max\big(0,\; |T_{G_k} \cup T_{G_{k+1}}| - C\big).$$

Moreover, this bound is achievable*: there exist configurations attaining equality.*

Proof. $\mathcal{C}_k \supseteq T_{G_k}$ and $\mathcal{C}_{k+1} \supseteq T_{G_{k+1}}$. Together they must cover $T_{G_k} \cup T_{G_{k+1}}$, but $|\mathcal{C}_{k+1}| = C$. Hence $\mathcal{C}_{k+1}$ can hold at most $C - |T_{G_{k+1}}|$ tools from $T_{G_k} \setminus T_{G_{k+1}}$, so at least $|T_{G_k} \setminus T_{G_{k+1}}| - (C - |T_{G_{k+1}}|) = |T_{G_k} \cup T_{G_{k+1}}| - C$ tools of T_{G_k} must leave the magazine, giving the lower bound.

Achievability. Construct $\mathcal{C}_k = T_{G_k} \cup X_k$ where $X_k \subseteq T_{G_{k+1}} \setminus T_{G_k}$ with $|X_k| = \min(C - |T_{G_k}|,\; |T_{G_{k+1}} \setminus T_{G_k}|)$ (fill all $s_k = C - |T_{G_k}|$ padding slots with tools of the *next* group). Similarly construct $\mathcal{C}_{k+1} = T_{G_{k+1}} \cup X_{k+1}$ with lookahead into G_{k+2}. Then

$$\mathcal{C}_k \setminus \mathcal{C}_{k+1} = \big(T_{G_k} \setminus \mathcal{C}_{k+1}\big) \cup \big(X_k \setminus \mathcal{C}_{k+1}\big).$$

Since $X_k \subseteq T_{G_{k+1}} \subseteq \mathcal{C}_{k+1}$, the second term is empty. Since $T_{G_k} \cap \mathcal{C}_{k+1} = T_{G_k} \cap T_{G_{k+1}}$ (no tool of T_{G_k} appears in the lookahead padding X_{k+1} of G_{k+2}, which is disjoint from T_{G_k} by design), we get $|\mathcal{C}_k \setminus \mathcal{C}_{k+1}| = |T_{G_k}| - |T_{G_k} \cap T_{G_{k+1}}| = |T_{G_k} \setminus T_{G_{k+1}}|$. This equals the lower bound $|T_{G_k} \cup T_{G_{k+1}}| - C$ when $|T_{G_{k+1}} \setminus T_{G_k}| \geq s_k$, i.e. when there are enough next-group tools to fill all padding slots, which holds whenever $|T_{G_k} \cup T_{G_{k+1}}| \geq C$ (the JGP-infeasibility condition for a single configuration). □ □

Remark 2.3. We call the construction in lemma 2.2 the **lookahead configuration**: $\mathcal{C}_k^* = T_{G_k} \cup X_k$ with $X_k \subseteq T_{G_{k+1}} \setminus T_{G_k}$, $|X_k| = s_k$ (all padding slots filled with next-group tools). When $s_k \geq |T_{G_{k+1}} \setminus T_{G_k}|$ the slack is insufficient to fully pre-load the next group; the actual transition cost is still $|T_{G_k} \setminus T_{G_{k+1}}|$ (mandatory outflow), which is the tightest achievable.

Construction I: Disjoint Private Tool Sets

We first study the natural candidate construction and show that disjoint private tool sets produce *no gap* — a structural insight that forces the correct design.

Construction 2.4 (Instance $\mathcal{I}(K, r, C)$ — Disjoint Private Sets)**.** Parameters: integers $K \geq 2$, $C/2 < r \leq C$ (so $2r > C$).
Tools: $T = P_1 \uplus P_2 \uplus \cdots \uplus P_K$ (pairwise disjoint), $|P_k| = r$ for each k, total $m = Kr$.
Groups: G_k consists of all jobs whose tool requirement is P_k, so $T_{G_k} = P_k$ and $|T_{G_k}| = r$.
Padding slack: $s_k = C - r$ for all k.
JGP-optimality: Since $|P_k \cup P_{k'}| = 2r > C$ for $k \neq k'$, no single configuration can serve two distinct groups; therefore $K^* = K$.

Naïve heuristic. Use $\hat{\mathcal{C}}_k = P_k \cup F$ where F is a *fixed* set of $C - r$ arbitrary tools disjoint from all P_k (shared filler). Then for any consecutive pair:

$$\hat{\mathcal{C}}_k \cap \hat{\mathcal{C}}_{k+1} = F, \qquad w(\hat{\mathcal{C}}_k, \hat{\mathcal{C}}_{k+1}) = |\hat{\mathcal{C}}_k \setminus \hat{\mathcal{C}}_{k+1}| = |P_k| = r, \qquad H = (K-1) \cdot r.$$

Optimal SSP with lookahead. Use $\mathcal{C}_k^* = P_k \cup X_k$, $X_k \subseteq P_{k+1}$, $|X_k| = C - r$. Then $\mathcal{C}_k^* \setminus \mathcal{C}_{k+1}^* = P_k$ (since $P_k \cap P_{k+1} = \emptyset$ and $P_k \cap X_{k+1} = \emptyset$), so the transition cost is still $|P_k| = r$.

Remark 2.5. For construction 2.4, *every* choice of valid configurations yields transition cost exactly r per consecutive pair (because P_k and P_{k+1} are disjoint, all r mandatory tools of P_k must leave the magazine). Hence $H = \mathrm{OPT}_{\mathrm{SSP}} = (K-1)r$ and the gap is **zero**. Disjoint private tool sets eliminate padding freedom as the source of the gap.

This negative result directs us to the correct construction: groups must share some mandatory tools for a non-trivial gap to exist.

Construction II: Overlapping Mandatory Tools — The Tight Construction

Construction 2.6 (Instance $\mathcal{I}(K, r, ov, C)$ — Overlapping Windows)**.** Parameters: integers $K \geq 2$, $0 \leq ov < r$, C with $2r - ov > C > r$ (ensures $K^* = K$ and groups are not full-capacity).
Tools: $T = \{t_1, t_2, \ldots, t_m\}$ with $m = (K-1)(r - ov) + r$.
Mandatory tool sets: $T_{G_k} = \{t_{(k-1)(r-ov)+1}, \ldots, t_{(k-1)(r-ov)+r}\}$ for $k = 1, \ldots, K$ (consecutive "windows" of width r, shifted by $r - ov$).
So $|T_{G_k}| = r$, $|T_{G_k} \cap T_{G_{k+1}}| = ov$, $|T_{G_k} \cup T_{G_{k+1}}| = 2r - ov$.
JGP-optimality: $2r - ov > C$ means no configuration covers two adjacent groups; $K^* = K$.
Padding slack: $s_k = C - r$ for all k.

Heuristic Cost (Naïve Padding)

Use $\hat{\mathcal{C}}_k = T_{G_k} \cup F_k$ where $F_k \subseteq T \setminus T_{G_{k+1}}$ (padding avoids all next-group mandatory tools), $|F_k| = s_k = C - r$. Then:

$$\hat{\mathcal{C}}_k \setminus \hat{\mathcal{C}}_{k+1} \supseteq T_{G_k} \setminus T_{G_{k+1}} = r - ov \text{ tools (mandatory exit)},$$
$$F_k \setminus \hat{\mathcal{C}}_{k+1} = F_k \text{ (all padding exits, since } F_k \cap T_{G_{k+1}} = \emptyset).$$

Therefore $w(\hat{\mathcal{C}}_k, \hat{\mathcal{C}}_{k+1}) = (r - ov) + (C - r) = C - ov$, giving

$$\boxed{H = (K-1)(C - ov).} \tag{1}$$

Optimal SSP Cost (Lookahead Configuration)

Use $\mathcal{C}_k^* = T_{G_k} \cup X_k$ where $X_k \subseteq T_{G_{k+1}} \setminus T_{G_k}$, $|X_k| = \min(s_k, r - ov) = \min(C - r, r - ov)$.
Assuming $ov \geq 2r - C$ (equivalently $C - r \leq r - ov$, so $s_k = C - r \leq r - ov$), we have $|X_k| = C - r$.

Transition cost computation.

$$\mathcal{C}_k^* \setminus \mathcal{C}_{k+1}^* = \big(T_{G_k} \setminus \mathcal{C}_{k+1}^*\big) \cup \big(X_k \setminus \mathcal{C}_{k+1}^*\big).$$

Since $X_k \subseteq T_{G_{k+1}} \subseteq \mathcal{C}_{k+1}^*$, the second term vanishes.

$$T_{G_k} \cap \mathcal{C}_{k+1}^* = T_{G_k} \cap T_{G_{k+1}} + T_{G_k} \cap X_{k+1} = ov + 0 = ov,$$

where $T_{G_k} \cap X_{k+1} = 0$ because $X_{k+1} \subseteq T_{G_{k+2}} \setminus T_{G_{k+1}}$ and $T_{G_k} \cap T_{G_{k+2}} = 0$ (non-adjacent windows do not overlap when $2(r - ov) > r$, i.e. $r > 2\,ov$; we may enforce this by choosing $ov < r/2$). Hence

$$|\mathcal{C}_k^* \setminus \mathcal{C}_{k+1}^*| = |T_{G_k}| - ov = r - ov,$$

and the total optimal cost for the path $G_1 \to G_2 \to \cdots \to G_K$ is

$$\boxed{\mathrm{OPT}_{\mathrm{SSP}} = (K-1)(r - ov).} \tag{2}$$

Proof That (2) Is Indeed the Optimum

We need to establish (2) rigorously, showing that no feasible SSP solution (over all groupings, all configurations, all orderings) achieves cost below $(K-1)(r - ov)$.

Theorem 2.7 (OPT-SSP for Construction II)**.** *For instance $\mathcal{I}(K, r, ov, C)$ with $ov \geq 2r - C$ and $r < C < 2r - ov$, the optimum SSP cost over all feasible groupings, configurations, and orderings is:*

$$\mathrm{OPT}_{SSP} = (K-1)(r - ov).$$

Proof. **Upper bound.** The lookahead solution (ordering $G_1, G_2, \ldots, G_K$ with configurations $\mathcal{C}_k^*$ as defined above) achieves cost $(K-1)(r - ov)$ by the computation in section 2.4.2. ✓

Lower bound. We argue that *any* feasible SSP solution — including solutions using more than K groups — incurs total switching cost at least $(K-1)(r - ov)$.

Consider the $K-1$ tool sets $D_k = T_{G_k} \setminus T_{G_{k+1}}$ for $k = 1, \ldots, K-1$. Each D_k has exactly $r - ov$ tools (the mandatory tools of window G_k that do not appear in window G_{k+1}). A key observation:

(i) Tool $t \in D_k$ is required by some job in G_k but by *no* job in $G_{k+1}, G_{k+2}, \ldots, G_K$ (it lies outside all later windows, since windows shift forward). Therefore, once all jobs of G_k are processed, tool t is never needed again.

(ii) Tool $t \in D_k$ is required by some job in G_k and *not* by any job in $G_1, \ldots, G_{k-1}$ (it enters the window only at step k). So t must be in the magazine at the time jobs of G_k are processed.

Now consider any feasible job sequence and any corresponding valid magazine schedule. Let $\tau(t)$ be the last point in the sequence at which tool t is used (i.e. the last job requiring t). After position $\tau(t)$, tool t may still remain in the magazine but will eventually be removed (or the sequence ends with t still loaded, which does not save a switch but may defer it).

For any tool $t \in D_k$: in the optimal SSP, tool t is last used somewhere in the sub-sequence of jobs from group G_k. It must be loaded into the magazine before or at the first job of G_k requiring it, and it must leave the magazine at some point after $\tau(t)$. Since t is never again needed, it must be removed (one switch attributed to t) at some transition.

The sets $D_1, D_2, \ldots, D_{K-1}$ are pairwise disjoint (each D_k consists of tools in window k but not window $k+1$, and the windows shift forward). Therefore the removal events for all tools across all D_k are distinct switch events. The total number of mandatory removals is $\sum_{k=1}^{K-1} |D_k| = (K-1)(r - ov)$.

This argument holds regardless of the grouping used: even if the solution uses $K' > K$ groups (a finer partition), every tool in $\bigcup_k D_k$ still exits the magazine exactly once after its last use, contributing at least $(K-1)(r - ov)$ switches in total. (A finer grouping cannot merge two D_k sets, since the tools in each D_k are structurally required by distinct job subsets

with disjoint later windows.)
Therefore $\mathrm{OPT}_{\mathrm{SSP}} \geq (K-1)(r-ov)$, completing the proof. □ □

The Tight Gap Bound

Theorem 2.8 (Tight Gap Bound)**.** *For instance $\mathcal{I}(K, r, ov, C)$ with $ov \geq 2r - C$, $r < C$, $2r - ov > C$, and $ov < r/2$:*

$$H - \mathrm{OPT}_{SSP} \;=\; (K-1)(C-r) \;=\; (K-1)\cdot s,$$

*where $s = C - r$ is the padding slack per group. This bound is **tight**.*

Proof. By (1) and (2): $H - \mathrm{OPT}_{\mathrm{SSP}} = (K-1)(C-ov) - (K-1)(r-ov) = (K-1)(C-r)$. Tightness: the construction achieves this gap exactly; no better fixed-padding heuristic can do better in the worst case, since the naïve padding is the worst-case choice and the lookahead achieves the optimum. □ □

Corollary 2.9 (Approximation Ratio)**.** *For the same instance family:*

$$\frac{H}{\mathrm{OPT}_{SSP}} = \frac{C - ov}{r - ov}.$$

*As $ov \to r - 1$ (near-complete overlap between adjacent groups), this ratio tends to $C - r + 1$. As $C \to \infty$ with r/C fixed and ov fixed, the ratio tends to ∞. Therefore **no constant approximation factor** is achievable for the JGP-grouping + naïve-padding TSP heuristic.*

Structural Insights and Summary of Part I

(1) **Gap is $(K^* - 1) \cdot s$.** The gap equals the number of inter-group transitions times the padding slack. It grows linearly in both K^* and $s = C - \max_k |T_{G_k}|$.
(2) **Disjoint private sets give zero gap.** When groups share no mandatory tools, there is no configuration freedom that the heuristic wastes: every transition costs exactly r regardless of padding. This confirms the known TSP-reduction for full-capacity jobs ($r = C$).
(3) **The gap is Source-1 only.** The lower-bound proof in theorem 2.7 shows that the K-group lookahead solution is already optimal: no finer grouping (more groups) can reduce the total switching count below $(K-1)(r-ov)$, because each of the $(K-1)(r-ov)$ mandatory tool removals must occur regardless of grouping. Hence Source 2 (grouping choice) contributes *zero* additional gap for this family; the entire gap comes from Source 1 (configuration choice).
(4) **Gap is eliminated by lookahead (GTSP).** If instead of naïve padding we solve GTSP over all valid configurations, the lookahead solution is found, achieving $\mathrm{OPT}_{\mathrm{SSP}}$ for the fixed grouping. This motivates the GTSP formulation in Part II.

Part II: Grouping Selection for Optimal and Heuristic SSP Solving

Given a pool of groupings — JGP-optimal ones ($K = K^*$), sub-optimal ones ($K > K^*$), and the trivial all-configurations-as-single-group option — we address how to select groupings to (a) solve SSP exactly, and (b) obtain strong heuristic solutions with quality guarantees.

Mathematical and Logical Directions

Reduction to Generalized TSP (GTSP) — The Exact Formulation

Theorem 3.1 (SSP for Fixed Grouping = GTSP)**.** *Given a fixed feasible grouping $\mathcal{G} = \{G_1, \ldots, G_K\}$, the problem of simultaneously selecting configurations $\mathcal{C}_k \in \mathfrak{C}(G_k)$ and ordering the groups to minimise total switching cost is equivalent to a Generalized Travelling Salesman Problem (GTSP) on K clusters.*

Proof. Define a complete weighted directed graph $\mathcal{H}$ as follows. For each group G_k create a cluster of nodes $V_k = \mathfrak{C}(G_k)$. For any $\mathcal{C}_i \in V_k$ and $\mathcal{C}_j \in V_{k'}$ $(k \neq k')$ set arc weight $w(\mathcal{C}_i, \mathcal{C}_j)$. A feasible SSP solution selects one configuration per group and sequences the groups; this corresponds to choosing exactly one node per cluster and finding a minimum-weight Hamiltonian path through the cluster-graph, which is by definition the GTSP. □ □

How to solve it. The Noon–Bean transformation converts GTSP on K clusters with N total nodes to an equivalent standard TSP on $O(N)$ nodes [8]. Solvers such as Concorde or LKH3 can then be applied. Alternatively, the Held–Karp DP adapted to clusters runs in $O(N^2 \cdot 2^K)$ time, which is tractable when $K = K^*$ is small.

Controlling cluster size. $|\mathfrak{C}(G_k)| = \binom{m-|T_{G_k}|}{s_k}$, which can be exponential. Two practical strategies:

(i) *Lookahead restriction*: only consider configurations of the form $\mathcal{C}_k = T_{G_k} \cup X_k$ where $X_k \subseteq T_{G_{k'}}$ for some adjacent group $G_{k'}$. This limits cluster size to $O(K)$ per group.

(ii) *Column generation within GTSP*: start with a small representative subset per cluster, solve, then add new configurations via a pricing oracle.

Guarantee. The GTSP solved to optimality for a fixed grouping $\mathcal{G}$ yields $\mathrm{OPT}(\mathcal{G})$, the best achievable SSP cost for that grouping. Taking the minimum over all candidate groupings gives $\mathrm{OPT}_{\mathrm{SSP}}$.

Grouping Selection via Tool-Overlap Graph

Definition 3.2 (Tool-Overlap Graph)**.** For a grouping $\mathcal{G} = \{G_1, \ldots, G_K\}$, define the *tool-overlap graph* H_{ov} as the complete weighted graph on K nodes (one per group) with edge weight $w(G_k, G_{k'}) = |T_{G_k} \cap T_{G_{k'}}|$.

Theorem 3.3 (Optimal Ordering Equivalence)**.** *Given a fixed grouping $\mathcal{G}$ with equal mandatory tool sizes ($|T_{G_k}| = r$ for all k), and assuming the lookahead configuration $\mathcal{C}_k^* = T_{G_k} \cup X_k$ (all slack filled with next-group tools) is used:*

$$\mathrm{OPT}(\mathcal{G}, \pi^*) = \sum_{k=1}^{K-1} (r - w(G_{\pi(k)}, G_{\pi(k+1)})),$$

where π^* is the optimal ordering. Since $\sum_k r$ is constant in π, minimising the SSP cost over all orderings π is equivalent to maximising *the total tool overlap along a Hamiltonian path in H_{ov}:*

$$\pi^* = \operatorname*{argmax}_{\pi} \sum_{k=1}^{K-1} |T_{G_{\pi(k)}} \cap T_{G_{\pi(k+1)}}|.$$

This Maximum-Weight Hamiltonian Path (MWHP) on K nodes is solvable in $O(K^2 \cdot 2^K)$ via Held–Karp DP.

Proof. Under the lookahead configuration, the transition cost from $G_{\pi(k)}$ to $G_{\pi(k+1)}$ is $|T_{G_{\pi(k)}} \setminus T_{G_{\pi(k+1)}}| = r - |T_{G_{\pi(k)}} \cap T_{G_{\pi(k+1)}}|$. Summing over $k = 1, \ldots, K-1$ and noting that $\sum_k r = r(K-1)$ is independent of π, minimising total cost is equivalent to maximising $\sum_k |T_{G_{\pi(k)}} \cap T_{G_{\pi(k+1)}}|$. □ □

Practical grouping-selection algorithm based on theorem 3.3.

1. Enumerate candidate groupings (JGP-optimal and a chosen set of suboptimal ones).
2. For each grouping $\mathcal{G}$, build H_{ov} and solve MWHP (tractable for small K).
3. Score each grouping by its MWHP value (higher = better).
4. Select the top-p groupings; solve GTSP exactly for each.
5. Return the best solution.

Lagrangian Relaxation — Exploring the Pareto Frontier

The core tension between JGP (minimise K) and SSP (minimise switches) is formalised via a Lagrangian relaxation. Define

$$L(\lambda) = \min_{\mathcal{G},\, \pi,\, \text{configs}} \Big[\text{SSP-cost}(\mathcal{G}, \pi) + \lambda \cdot (|\mathcal{G}| - K^*)\Big], \quad \lambda \geq 0.$$

$L(\lambda)$ is a valid lower bound on OPT_{SSP} for all $\lambda \geq 0$ (by weak Lagrangian duality), and $L(\lambda)$ is concave and piecewise linear in λ.

Properties.

- $\lambda = 0$: unconstrained SSP (any K allowed).
- $\lambda \to \infty$: forces $|\mathcal{G}| = K^*$ (JGP-constrained SSP).
- The optimal $\lambda^* = \text{argmax}_{\lambda \geq 0} L(\lambda)$ gives the tightest Lagrangian lower bound $L(\lambda^*) \leq \text{OPT}_{\text{SSP}}$.

Subgradient algorithm.

1. Initialise $\lambda^{(0)} = 0$; step sizes $\alpha^{(t)} = \alpha_0/\sqrt{t}$.
2. At iteration t: solve $L(\lambda^{(t)})$ for the optimal grouping $\mathcal{G}^{(t)}$ (this is a penalised SSP, decomposable as GTSP on $|\mathcal{G}^{(t)}|$ groups).
3. Subgradient: $g^{(t)} = |\mathcal{G}^{(t)}| - K^*$.
4. Update: $\lambda^{(t+1)} = \max\big(0,\, \lambda^{(t)} + \alpha^{(t)} g^{(t)}\big)$.
5. Track best primal feasible solution across all iterations.
6. Stop when $|g^{(t)}| < \varepsilon$ or iteration budget exhausted.

The sequence $\{\mathcal{G}^{(t)}\}$ traces the Pareto frontier of (number of groups, switch cost), directly providing sub-optimal JGP groupings that may achieve lower switching cost.

> **Proposition 3.4** (Lagrangian Bound Quality)**.** *$L(\lambda^*) \leq \text{OPT}_{SSP} \leq L(\lambda^*) + \Delta_{\text{IP}}$, where Δ_{IP} is the integrality gap of the LP relaxation of the joint grouping-sequencing problem. Empirically, Δ_{IP} is small ($< 5\%$) on practical FMS instances.*

Column Generation — The Exact Master Problem

Model SSP as a set-partition ILP over all feasible groupings Γ:

$$\begin{aligned} \min_{\mathbf{y}} \quad & \sum_{\mathcal{G} \in \Gamma} c(\mathcal{G})\, y_{\mathcal{G}} && (3) \\ \text{s.t.} \quad & \sum_{\mathcal{G} \in \Gamma:\, j \in \mathcal{G}} y_{\mathcal{G}} = 1, \quad \forall\, j \in J && \text{(each job covered once)} \\ & y_{\mathcal{G}} \in \{0, 1\}, \quad \forall\, \mathcal{G} \in \Gamma. \end{aligned}$$

Here $c(\mathcal{G}) = \text{OPT}(\mathcal{G})$ is the optimal GTSP cost for grouping $\mathcal{G}$.

Column generation procedure.

1. Maintain a Restricted Master Problem (RMP) with an initial small set of groupings.
2. Solve RMP LP; obtain dual variables u_j (one per job).
3. *Pricing problem*: find $\mathcal{G} \in \Gamma$ with reduced cost $c(\mathcal{G}) - \sum_{j \in \mathcal{G}} u_j < 0$. This is a modified JGP where jobs carry dual-price values; solvable by Lagrangian-weighted branch-and-bound.
4. If such $\mathcal{G}$ exists, add to RMP and re-solve; otherwise, LP is optimal.
5. Apply branch-and-bound on the integer variables for the ILP.

Guarantees. The LP relaxation of (3) provides a certified lower bound on $\mathrm{OPT}_{\mathrm{SSP}}$ at every iteration. Branch-and-price yields the exact optimum; the LP bound at any intermediate iteration gives an anytime optimality gap.

Tool-Conflict Hypergraph and Chromatic Structure

Definition 3.5 (Tool Conflict Graph)**.** $G_c = (J, E_c)$ where $(j, j') \in E_c$ iff $|T_j \cup T_{j'}| > C$ (jobs j, j' cannot share a configuration).

A valid grouping is a proper K-colouring of G_c (each colour class is a feasible group). Properties:

- The minimum K is related to the clique number $\omega(G_c)$: $K^* \geq \lceil \omega(G_c) / \lfloor C / \max_j |T_j| \rfloor \rceil$.
- Each colour class (group) is a clique in the complement graph $\overline{G_c}$ (jobs sharing a configuration form an independent set in G_c).

Grouping selection criterion. Among all K^*-colourings of G_c, prefer those where adjacent colour classes (in the SSP sequence) have maximum mandatory tool overlap. This is formally:

$$\max_{\text{colouring}} \max_{\pi} \sum_{k=1}^{K-1} |T_{G_{\pi(k)}} \cap T_{G_{\pi(k+1)}}|,$$

a joint colouring-and-ordering optimisation. This unified graph-theoretic view characterises the best grouping in terms of the chromatic structure of G_c.

Machine Learning Directions

Graph Neural Network (GNN) Grouping Scorer

Motivation and architecture. The goal is to score a large pool of candidate groupings rapidly — without solving GTSP for each. GNNs are architecturally appropriate: the SSP cost is *permutation-invariant* in the groups, and GNNs are permutation-equivariant.

Input graph. Bipartite job–tool graph $B = (J \cup T, E)$, where $(j, t) \in E$ iff $t \in T_j$. Node features: $|T_j|$ for job nodes; frequency $\sum_j \mathbf{1}[t \in T_j]$ for tool nodes.

Job encoding. Two-layer Graph Attention Network (GAT) or GraphSAGE producing job embeddings $h_j \in \mathbb{R}^d$.

Group representation. For grouping $\mathcal{G} = \{G_1, \ldots, G_K\}$: $\mathbf{g}_k = \frac{1}{|G_k|} \sum_{j \in G_k} h_j$ (mean pool).

Grouping-level graph. Complete graph on K nodes with node features $\mathbf{g}_k$ and edge features $|T_{G_k} \cap T_{G_{k'}}|$.

Output. Second GNN (or Transformer) on the K-node graph predicts $\mathrm{OPT}(\mathcal{G})$ as a scalar.

Mathematical justification. By the universality theorem for GNNs [9], GNNs with sufficient depth and width can represent any function on graphs. Since $\mathrm{OPT}(\mathcal{G})$ depends only on the tool-overlap structure of the grouping (captured by the K-node grouping graph), the GNN can in principle represent it exactly. Empirically, 2–3 layers suffice for moderate K.

Training. Generate training data by solving instances of moderate size exactly (GTSP for each grouping). Include both JGP-optimal ($K = K^*$) and suboptimal ($K > K^*$) groupings to ensure the model learns across the full Pareto frontier. Minimise MSE loss $\mathbb{E}[|\hat{c}(\mathcal{G}) - \mathrm{OPT}(\mathcal{G})|^2]$.

Inference procedure.

1. Generate all JGP-optimal groupings plus a selected set of suboptimal ones.
2. Score all groupings using the GNN in $O(K^2)$ per grouping.
3. Select top-p groupings; solve GTSP exactly for each.
4. Return the best solution found.

Probabilistic approximation guarantee. If the GNN approximates $\mathrm{OPT}(\mathcal{G})$ within additive error ε with probability $\geq 1 - \delta$ over the training distribution $\mathcal{D}$, then the selected grouping has cost $\leq \mathrm{OPT}_{\mathrm{SSP}} + \varepsilon + \eta$, where $\eta \to 0$ as the candidate pool and p increase.

Reinforcement Learning for Joint Grouping and Sequencing

RL jointly optimises grouping and sequencing, naturally exploring $K > K^*$ and discovering when more groups reduce total switches.

MDP formulation.

State S_t: partial assignment of jobs to groups; current partial schedule.

Action A_t: assign next unassigned job to an existing group or open a new one.

Transition: deterministic.

Reward: $R_T = -\,\mathrm{OPT}_{\mathrm{SSP}}(\text{final grouping})$ at terminal state; $R_t = 0$ for $t < T$.

Policy π_θ: Transformer or Pointer Network.

Training (REINFORCE with baseline).

$$\nabla_\theta \mathcal{L}(\theta) = -\mathbb{E}_\pi \left[R_T \cdot \sum_t \nabla_\theta \log \pi_\theta(A_t \mid S_t) \right].$$

Variance is reduced using a greedy rollout baseline or a learned value network $V_\phi(S_t) \approx \mathbb{E}[R_T \mid S_t]$ (Actor–Critic variant).

Architecture (following Kool et al. [11]). Encode each job j as $f_j = [|T_j|, \phi_j]$ where $\phi_j \in \mathbb{R}^m$ is the binary tool-requirement vector. Multi-head self-attention produces context-aware representations. A decoder with masked attention selects group assignments; invalid actions (exceeding capacity) are masked out.

Guarantee. No worst-case approximation ratio is guaranteed. Distributional guarantee: $\mathbb{E}_{I\sim\mathcal{D}}[\mathrm{cost}(\pi_\theta(I))] \to \mathbb{E}_{I\sim\mathcal{D}}[\mathrm{OPT}_{\mathrm{SSP}}(I)]$ as training progresses under standard regularity conditions.

Learning-Augmented Column Generation

This is the most theoretically sound ML approach: it *preserves exact optimality* while using ML to accelerate the pricing step.

ML augmentation of the pricing oracle. Train a predictor $P_\phi : (u, B) \to \mathcal{G}'$ mapping dual variables $u \in \mathbb{R}^n$ and the instance bipartite graph B to a candidate grouping $\mathcal{G}'$. At each CG iteration:

(a) Query P_ϕ to obtain candidate $\mathcal{G}'$.
(b) Verify: compute $c(\mathcal{G}') - \sum_j u_j$ (cheap). If negative, add $\mathcal{G}'$ to RMP.
(c) If P_ϕ fails (positive reduced cost), fall back to exact pricing solver.

Architecture for P_ϕ. GNN on B produces job embeddings; concatenate with u_j per job; graph-partitioning head outputs a grouping assignment. Train on $(u^*, \mathcal{G}^*)$ pairs from previously solved instances.

Correctness guarantee (exact optimality preserved). Every ML-suggested column is verified before acceptance; the fallback exact solver certifies LP optimality. ML only reduces the number of exact solver calls needed, not the solution quality. Empirically, learning-augmented CG reduces pricing iterations by 50–80% on analogous combinatorial problems [10, 12].

Contrastive Learning for Grouping Comparison

A lightweight alternative: train a model to compare pairs of groupings (which is better?) rather than predict absolute costs.

Training data: pairs $(\mathcal{G}_A, \mathcal{G}_B)$ with label 1 if $\mathrm{OPT}(\mathcal{G}_A) < \mathrm{OPT}(\mathcal{G}_B)$, else 0.

Model: Siamese GNN — same encoder applied to both groupings, comparison head.

Loss: binary cross-entropy.

Inference: given N candidates, run a tournament ($O(N \log N)$ comparisons) to identify the best grouping; solve GTSP only for the top-ranked candidate.

This scales to very large candidate pools and requires only relative quality labels, which are easier to obtain than exact GTSP costs.

Part III: SSP Variants That Collapse to JGP

We characterise SSP variants — modified by a pairwise switching cost function f — for which solving SSP-f is equivalent to solving JGP: the optimal SSP-f solution always uses exactly K^* groups.

Formal Definition of Collapse

Definition 4.1 (Collapse to JGP)**.** SSP-f (SSP with switching cost function $f : 2^T \times 2^T \to \mathbb{R}_+$) *collapses to JGP* if for every instance I:

$$\operatorname*{argmin}_{\mathcal{G},\,\pi} \mathrm{SSP}\text{-}f(I, \mathcal{G}, \pi) \;\subseteq\; \big\{\mathcal{G} : |\mathcal{G}| = K^*(I)\big\},$$

i.e. the optimal SSP-f solution always uses exactly K^* groups.

The Baseline Collapse: Switching Instants (0/1 Cost)

Define $f_{01}(\mathcal{C}, \mathcal{C}') = \mathbf{1}[\mathcal{C} \neq \mathcal{C}']$. This is the Tool Switching Instants Problem (TSIP). Under TSIP, total cost = number of configuration changes = $K - 1$ for K distinct configurations in a path. Minimising this equals minimising K = JGP. This collapse is exact and trivial, but TSIP discards all information about which tools change — it is not a metrically rich variant. We seek richer ones.

Threshold SSP

Definition 4.2 (Threshold SSP)**.** For parameter $\theta \geq 0$, define $f_\theta(\mathcal{C}, \mathcal{C}') = \max(0,\ |\mathcal{C} \setminus \mathcal{C}'| - \theta)$. Switching up to θ tools between consecutive configurations is "free"; only excess switches incur cost.

Theorem 4.3 (Threshold Collapse Condition)**.** *Let $\mathcal{G}^* = \{G_1, \ldots, G_K\}$ be any JGP-optimal grouping with lookahead configurations $\mathcal{C}_k^*$. The transition cost under f_θ is $f_\theta(\mathcal{C}_k^*, \mathcal{C}_{k+1}^*) = \max(0,\ (r - ov_k) - \theta)$, where $ov_k = |T_{G_k} \cap T_{G_{k+1}}|$.*

Define $\theta^* = \max_{k=1,\ldots,K-1} |T_{G_k} \setminus T_{G_{k+1}}| = \max_k(r_k - ov_k)$ (maximum mandatory outflow over all consecutive pairs, using the lookahead configuration). Then:

(i) For $\theta \geq \theta^*$: SSP-$f_\theta(\mathcal{G}^*, \pi, \{\mathcal{C}_k^*\}) = 0$. Since $f_\theta \geq 0$ always, $\mathrm{OPT}_{\text{SSP-}f_\theta} = 0$, achieved by any JGP-optimal grouping. SSP-f_θ collapses to JGP.

(ii) For $0 \leq \theta < \theta^*$: SSP-f_θ interpolates between standard SSP ($\theta = 0$) and JGP ($\theta = \theta^*$), with strictly positive optimal cost.

Proof. (i): For each k, the lookahead transition cost is $|T_{G_k} \setminus T_{G_{k+1}}| = r_k - ov_k \leq \theta^* \leq \theta$. Hence $f_\theta(\mathcal{C}_k^*, \mathcal{C}_{k+1}^*) = \max(0, (r_k - ov_k) - \theta) = 0$. Summing over k gives total cost $= 0$. Since $f_\theta \geq 0$, this is optimal. Any JGP-optimal grouping achieves this, so the SSP-f_θ solution set contains all JGP-optimal groupings.

(ii): For $\theta < \theta^*$, there exists some pair k^* with $|T_{G_{k^*}} \setminus T_{G_{k^*+1}}| > \theta$. Under any valid configuration for that pair, the transition cost is $\geq (|T_{G_{k^*}} \setminus T_{G_{k^*+1}}| - \theta) > 0$. Therefore $\mathrm{OPT}_{\text{SSP-}f_\theta} > 0$ and SSP-f_θ does not trivially collapse to JGP (the SSP problem remains non-trivial). □ □

Remark 4.4. The collapse threshold θ^* depends on the specific JGP-optimal grouping chosen (since ov_k varies across groupings). Across all JGP-optimal groupings: $\theta^*_{\min} \leq \theta^* \leq \theta^*_{\max}$. Using the grouping with the highest tool overlap (i.e. the one selected by theorem 3.3) minimises θ^*, making collapse easier to achieve.

Per-Tool Weight Variant

Definition 4.5 (Weighted SSP)**.** Let $w : T \to \mathbb{R}_{>0}$ assign a weight to each tool. Define $f_w(\mathcal{C}, \mathcal{C}') = \sum_{t \in \mathcal{C} \setminus \mathcal{C}'} w_t$.

Theorem 4.6 (Per-Tool Weight Collapse Condition)**.** *SSP-f_w collapses to JGP if and only if: for every pair of adjacent groups (G_k, G_{k+1}) in the optimal ordering and for every valid configuration $\mathcal{C} \in \mathfrak{C}(G_k)$,*

$$\sum_{t \in \mathcal{C} \setminus \mathcal{C}_{k+1}^*} w_t \ \geq\ \delta > 0,$$

where $\mathcal{C}_{k+1}^*$ is the optimal configuration for G_{k+1}, and δ exceeds the maximum possible cost reduction from splitting any group. This is the no-beneficial-split *condition.*

Proof. Necessity: if for some pair the transition cost under f_w is 0, then splitting G_k into two sub-groups adds one transition of cost 0 without increasing cost — the SSP-f_w optimum may use $K^* + 1$ groups, violating collapse.

Sufficiency: if every inter-group transition costs $\geq \delta > 0$, then any solution with $K > K^*$ groups incurs at least $K - 1 \geq K^*$ transitions, each costing $\geq \delta$. A JGP-optimal solution incurs $K^* - 1$ transitions, also each costing $\geq \delta$ but with *fewer* transitions. For the no-beneficial-split

condition (δ large enough), the total cost of any $K > K^*$ solution exceeds that of the K^*-group solution with optimised configurations. □ □

General Collapse Characterisation

Theorem 4.7 (General Collapse Characterisation)**.** *Let $f : 2^T \times 2^T \to \mathbb{R}_+$. SSP-$f$ collapses to JGP (on all instances) if and only if the following three conditions hold simultaneously:*

(C1) Self-cost zero: *$f(\mathcal{C}, \mathcal{C}) = 0$ for all $\mathcal{C} \subseteq T$.*

(C2) Strict inter-group positivity: *For any JGP-optimal grouping $\mathcal{G}^*$ with K^* groups, any two distinct groups $G_k \neq G_{k'}$, and any valid configurations $\mathcal{C} \in \mathfrak{C}(G_k)$, $\mathcal{C}' \in \mathfrak{C}(G_{k'})$: $f(\mathcal{C}, \mathcal{C}') \geq \delta_{\min} > 0$.*

(C3) Beneficial-split impossibility: *For any grouping $\mathcal{G}$ with $|\mathcal{G}| > K^*$: $\mathrm{OPT}_f(\mathcal{G}) \geq \mathrm{OPT}_f(\mathcal{G}^*)$ (using more groups cannot reduce cost).*

Proof. **Necessity.** ¬(C1): If $f(\mathcal{C}, \mathcal{C}) > 0$ for some $\mathcal{C}$, then even the trivial 1-group solution (if $K^* = 1$) incurs positive cost for each "non-transition". This contradicts K^*-optimality (a 0-group solution doesn't exist; but 1-group incurs 0 transitions, so cost = 0 under (C1)). Formally, (C1) is required so that within-group job ordering incurs zero cost. ✓

¬(C2): If $f(\mathcal{C}, \mathcal{C}') = 0$ for some inter-group pair, splitting group G_k into G'_k and G''_k adds one transition of cost 0, giving a $(K^* + 1)$-group solution with the same cost. SSP-f optimal may use $K^* + 1$ groups, violating definition 4.1. ✓

¬(C3): By definition, if adding groups reduces cost, the SSP-f optimal uses $> K^*$ groups. ✓

Sufficiency. Under (C1), the cost of a grouping with K groups and optimal ordering is $\geq (K-1)\delta_{\min}$ by (C2) (at least $K-1$ inter-group transitions, each costing $\geq \delta_{\min}$). Under (C3), the cost of any $K > K^*$ grouping is $\geq$ the cost of the K^*-group optimum. Therefore the SSP-f optimal uses exactly K^* groups. □ □

A Novel Variant: Setup-Matrix SSP

Definition 4.8 (Setup-Matrix SSP)**.** Let $W \in \mathbb{R}_+^{m \times m}$ with $W_{tt} = 0$ and $W_{tt'} \geq 0$ for $t \neq t'$. Define

$$f_W(\mathcal{C}, \mathcal{C}') = \sum_{t \in \mathcal{C} \setminus \mathcal{C}'} \min_{t' \in \mathcal{C}' \setminus \mathcal{C}} W_{tt'}.$$

Here $W_{tt'}$ is the cost of removing tool t when tool t' is being loaded in its place. This models sequence-dependent slot reuse costs.

Theorem 4.9 (Setup-Matrix Collapse Condition)**.** *If W satisfies $W_{tt'} \geq \gamma > 0$ for all $t \neq t'$ (uniform lower bound on all cross-tool setup costs), then:*

$$f_W(\mathcal{C}_k, \mathcal{C}_{k+1}) \geq \gamma \cdot |\mathcal{C}_k \setminus \mathcal{C}_{k+1}| \geq \gamma \cdot (2r - C)$$

for any consecutive group pair in any K^*-optimal grouping (using lemma 2.2). SSP-f_W satisfies (C2) of theorem 4.7 with $\delta_{\min} = \gamma(2r - C)$ and collapses to JGP when (C3) also holds, specifically when:

$$\gamma \cdot (2r - C) \;>\; \textit{maximum possible cost reduction from splitting one group},$$

i.e. the per-group-boundary cost exceeds any savings from finer partitioning.

Proof. By definition of f_W: each of the $|\mathcal{C}_k \setminus \mathcal{C}_{k+1}|$ removed tools t incurs cost $\geq \gamma \cdot 1 = \gamma$ (since $\min_{t'} W_{tt'} \geq \gamma$ for $t \neq t'$ and there is at least one $t' \in \mathcal{C}_{k+1} \setminus \mathcal{C}_k$ because $|\mathcal{C}_{k+1} \setminus \mathcal{C}_k| = |\mathcal{C}_k \setminus \mathcal{C}_{k+1}|$ by the uniform-SSP constraint). The second inequality follows from lemma 2.2. Condition (C3) becomes a numerical condition on γ relative to instance parameters, and can be certified for any specific instance. □ □

Machine Learning for Collapse Variants

Learning the Collapse Threshold θ^*

Rather than computing θ^* analytically (which requires solving JGP first), train a regression model to predict it from instance statistics:

- **Features**: $(n, m, C, \mathbb{E}[|T_j|], \mathrm{Var}[|T_j|], \max_j |T_j|, \rho)$ where ρ is tool co-occurrence density.
- **Target**: $\theta^*(I)$.
- **Architecture**: MLP or gradient-boosted trees (low-dimensional feature space).
- **Training**: solve a set of instances exactly; record θ^* from JGP solutions.

At inference: predict $\hat{\theta}^*$; solve SSP-$f_{\hat{\theta}^*}$; the solution is JGP-optimal (zero cost), providing a fast certificate of optimality.

Meta-Learning a Collapse-Inducing Cost Function

Definition 4.10 (Meta-Learning Objective)**.** Learn a parameterised cost function $f_\phi = f_{W_\phi}$ (e.g. the Setup-Matrix variant with learned W_ϕ) that simultaneously:

(i) makes SSP-f_ϕ collapse to JGP (SSP-f_ϕ-optimal solutions use K^* groups), and
(ii) preserves near-optimal solution quality for the original SSP.

This is formulated as a bilevel optimisation:

$$\begin{aligned} \min_{\phi} \quad & \sum_I \left|\mathrm{OPT}_{\text{SSP-}f_\phi}(I) - \mathrm{OPT}_{\text{SSP}}(I)\right| + \lambda\, R(\phi) \\ R(\phi) = & \sum_I \max\big(0,\, \mathrm{OPT}_{f_\phi}(I, K \leq K^*(I)) - \mathrm{OPT}_{f_\phi}(I, K = K^*(I))\big), \end{aligned} \tag{4}$$

where $R(\phi) = 0$ iff SSP-f_ϕ collapses to JGP on all training instances.

Training algorithm.

1. Initialise $W_\phi = \gamma I$ (uniform costs, $\gamma > 0$).
2. For each training instance I: compute $\mathrm{OPT}_{\text{SSP-}W_\phi}(I)$ and the regulariser $R_I(\phi)$.
3. Gradient update: $\phi \leftarrow \phi - \eta \cdot \nabla_\phi \big[\sum_I |\mathrm{OPT}_{\text{SSP-}f_\phi}(I) - \mathrm{OPT}_{\text{SSP}}(I)| + \lambda R_I(\phi)\big]$.
4. Iterate until convergence.

Theoretical justification. This is *surrogate optimisation*: the original hard SSP objective is replaced by SSP-f_ϕ, which is easier to optimise (by collapsing to JGP) but whose optima are close to those of the original SSP (enforced by the first term in (4)). The collapse constraint (enforced by $R(\phi)$) ensures tractability; the fidelity term ensures quality.

Instance-Specific Lagrangian Parameter Learning

From section 3.1.3, the Lagrangian parameter λ^* characterises the SSP/JGP tradeoff for each instance. Rather than running the subgradient algorithm from scratch, learn a model $\lambda^*(I) = f_\psi(\text{features}(I))$:

- **Input**: instance features (as in section 4.7).
- **Output**: $\hat{\lambda}^*(I)$.
- **Training**: record λ^* from subgradient runs on training instances.

At inference: predict $\hat{\lambda}^*$, solve $L(\hat{\lambda}^*)$ in one pass, recover near-optimal primal solution. This eliminates the iterative subgradient procedure and is analogous to warm-starting MIP solvers [10], achieving 50–90% solve-time reductions in related settings.

Summary of Key Results

Table 1: Summary of key results across all three parts.

Part	Key Result	Mathematical Content	Status
I	Gap $H - \mathrm{OPT}_{\mathrm{SSP}} = (K^* - 1) \cdot s$ (tight)	Construction II with $ov \geq 2r - C$; proven via KTNS counting lower bound	Novel; tight
I	Approx. ratio $H/\mathrm{OPT} = (C - ov)/(r - ov)$ is unbounded	$\to \infty$ as $ov \to r - 1$ or $C \to \infty$	No const. guarantee
I	Gap $= 0$ when $r = C$ (no padding slack)	SSP reduces to TSP; confirmed by construction	Known, confirmed
I	Disjoint private sets give zero gap	All transitions cost r regardless of padding; new structural insight	Novel insight
II-A1	SSP for fixed grouping = GTSP	Noon–Bean; Held–Karp in $O(N^2 \cdot 2^K)$	Exact formulation
II-A2	Optimal ordering = Max-Weight Ham. Path on overlap graph	theorem 3.3; constant sum argument	Novel equivalence
II-A3	Lagrangian explores K vs. switch Pareto frontier	Subgradient; $L(\lambda^*) \leq \mathrm{OPT} \leq L(\lambda^*) + \Delta_{\mathrm{IP}}$	Novel application
II-A4	Column generation for exact SSP	Set-partition master; pricing = modified JGP	Exact; branch-and-price
II-B1	GNN grouping scorer; perm. equivariant	GNN universality; bipartite job–tool encoding	Justified ML
II-B2	RL for joint grouping+sequencing	REINFORCE; Transformer/Pointer Net; explores $K > K^*$	Distributional guarantee
II-B3	Learning-augmented CG; exact optimality preserved	ML pricing predictor; fallback solver	Exact + ML speedup
II-B4	Contrastive learning for grouping ranking	Siamese GNN; tournament inference	Scalable heuristic
III.2	TSIP collapses to JGP	$f = \mathbf{1}[\mathcal{C} \neq \mathcal{C}']$; baseline	Known
III.3	Threshold SSP collapses for $\theta \geq \theta^*$	theorem 4.3; $\theta^* = \max_k \lvert T_{G_k} \setminus T_{G_{k+1}} \rvert$	Novel theorem
III.5	General collapse iff (C1)+(C2)+(C3) hold	theorem 4.7; necessary and sufficient	Novel characterisation
III.6	Setup-Matrix SSP and collapse condition	theorem 4.9; γ-lower bound on W	Novel variant
III.7	Meta-learning f^* that collapses to JGP	Bilevel optimisation; surrogate objective	Open; novel direction

Open Questions

Q1. (Part I) Is there a polynomial-time verifiable structural property of a grouping $\mathcal{G}$ that certifies $H(\mathcal{G}) - \mathrm{OPT}_{\mathrm{SSP}} \leq \varepsilon$? The maximum-overlap Hamiltonian path score (theorem 3.3)

is the most promising candidate. Proving tight approximation ratios for groupings selected by this criterion is open.

Q2. (Part II) What is the complexity of the pricing problem in column generation for SSP? Is it fixed-parameter tractable in K^*? This determines practical feasibility of branch-and-price.

Q3. (Part II) Can the GTSP cluster sizes be bounded polynomially for structured instances (e.g. when tool sets form a laminar family or an interval structure)? Such structure would make the exact formulation of theorem 3.1 directly practical.

Q4. (Part III) Does there exist a cost function f that simultaneously (a) makes SSP-f polynomial-time solvable, (b) collapses to JGP, and (c) preserves $\mathrm{OPT}_{\mathrm{SSP}}$ within a constant factor? This would be a major result; current evidence suggests no such f exists in general (likely implying $\mathrm{P} = \mathrm{NP}$), but a parameterised version (fixed-parameter tractable in K^* or $m - C$) may be achievable.

Q5. (Part III/ML) Convergence guarantees and sample complexity bounds for the meta-learning bilevel optimisation (4) are completely open. The landscape of $R(\phi)$ as a function of ϕ is non-convex and potentially discontinuous; analysing its gradient structure is an open theoretical question.

References

References

[1] C.S. Tang and E.V. Denardo, "Models arising from a flexible manufacturing machine," *Operations Research*, vol. 36, no. 5, pp. 767–784, 1988. [Original SSP paper; KTNS algorithm.]

[2] Y. Crama, O.E. Flippo, J. van de Klundert, and F.C.R. Spieksma, "The tool switching problem revisited," *European Journal of Operational Research*, vol. 74, no. 2, pp. 331–342, 1994. [NP-hardness proof.]

[3] Y. Crama and J. van de Klundert, "Worst-case performance of approximation algorithms for tool management problems," *Naval Research Logistics*, vol. 46, no. 5, pp. 445–462, 1999. [Approximation ratio analysis.]

[4] G. Laporte, J.J. Salazar-González, and F. Semet, "Exact algorithms for the job sequencing and tool switching problem," *IIE Transactions*, vol. 36, no. 1, pp. 37–45, 2004. [ILP + branch-and-bound.]

[5] D. Catanzaro, L. Gouveia, and M. Labbé, "Improved ILP formulations for the job sequencing and tool switching problem," *European Journal of Operational Research*, vol. 244, no. 3, pp. 766–777, 2015. [Tighter LP relaxation; current state-of-the-art ILP.]

[6] T.T. da Silva, A.A. Chaves, L.A.N. Lorena, and L.C. Coelho, "Multicommodity flow formulation and column generation for the SSP," 2021. [LP lower bound $\mathrm{OPT}_{\mathrm{SSP}} \geq m - C$.]

[7] G. Ghiani, A. Grieco, and R. Musmanno, "Reformulation and solution of the tool switching problem as a least-cost Hamiltonian cycle problem," *Journal of Intelligent Manufacturing*, vol. 18, no. 4, pp. 523–531, 2007. [Hamiltonian cycle formulation.]

[8] C.E. Noon and J.C. Bean, "An efficient transformation of the generalized traveling salesman problem," *INFORMS Journal on Computing*, vol. 5, no. 3, pp. 571–575, 1993. [Noon–Bean transformation: GTSP to TSP.]

[9] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" *Proceedings of ICLR 2019*, 2019. [GNN universality theorem.]

[10] E. Khalil, H. Dai, Y. Zhang, B. Dilkina, and L. Song, "Learning combinatorial optimization algorithms over graphs," *Advances in NeurIPS*, 2016. [Learning-augmented combinatorial optimisation.]

[11] W. Kool, H. van Hoof, and M. Welling, "Attention, learn to solve routing problems!" *Proceedings of ICLR 2019*, 2019. [Transformer/Attention for TSP; basis for RL direction.]

[12] M. Gasse, D. Chételat, N. Ferroni, L. Charlin, and A. Lodi, "Exact combinatorial optimisation with graph convolutional neural networks," *Advances in NeurIPS*, 2019. [GNN for branch-and-bound / learning-augmented CG.]